

CartService.java

```
package com.harisudhan.harisudhanmart.service;

import com.harisudhan.harisudhanmart.dao.CartDAO;
import com.harisudhan.harisudhanmart.dao.ProductDAO;
import com.harisudhan.harisudhanmart.exception.NotFoundException;
import com.harisudhan.harisudhanmart.exception.ValidationException;
import com.harisudhan.harisudhanmart.model.CartItem;
import com.harisudhan.harisudhanmart.model.Product;
import java.math.BigDecimal;
import java.sql.SQLException;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;

public class CartService {

    private final CartDAO cartDAO;
    private final ProductDAO productDAO;

    public CartService(CartDAO cartDAO, ProductDAO productDAO) {
        this.cartDAO = cartDAO;
        this.productDAO = productDAO;
    }

    public CartItem addItem(long userId, long productId, int quantity)
        throws ValidationException, NotFoundException, SQLException {
        if (quantity <= 0) {
            throw new ValidationException("quantity", "Quantity must be at least 1");
        }
        Product product = productDAO.findById(productId)
            .orElseThrow(() -> new NotFoundException("Product not found"));
        if (product.getStockQty() < quantity) {
            throw new ValidationException("quantity", "Not enough stock available");
        }
        CartItem item = new CartItem();
        item.setUserId(userId);
        item.setProductId(productId);
        item.setQuantity(quantity);
        return cartDAO.upsert(item);
    }

    public void updateQuantity(long userId, long cartItemId, int quantity) throws ValidationException, SQLException {
        if (quantity <= 0) {
            throw new ValidationException("quantity", "Quantity must be at least 1");
        }
        cartDAO.updateQuantity(cartItemId, userId, quantity);
    }

    public void removeItem(long userId, long cartItemId) throws SQLException {
        cartDAO.remove(cartItemId, userId);
    }

    /** Returns the cart lines paired with live product data, plus the running total. */
    public CartView viewCart(long userId) throws SQLException {
        List<CartItem> items = cartDAO.findByUser(userId);
        Map<CartItem, Product> lines = new LinkedHashMap<>();
        BigDecimal total = BigDecimal.ZERO;
        for (CartItem item : items) {
            Product product = productDAO.findById(item.getProductId()).orElse(null);
            if (product == null) {
                continue;
            }
            lines.put(item, product);
        }
    }
}
```

```
        total = total.add(product.getPrice().multiply(BigDecimal.valueOf(item.getQuantity())));
    }
    return new CartView(lines, total);
}

public void clear(long userId) throws SQLException {
    cartDAO.clear(userId);
}

public static class CartView {
    private final Map<CartItem, Product> lines;
    private final BigDecimal total;

    public CartView(Map<CartItem, Product> lines, BigDecimal total) {
        this.lines = lines;
        this.total = total;
    }

    public Map<CartItem, Product> getLines() { return lines; }
    public BigDecimal getTotal() { return total; }
}
}
```